Rapport de Projet SGBD

Gestion de parkings et flux de véhicules

Réalisé par

El Yaktini Sohayb

Bel Hajjam Yahya

Ajaji Mohamed

Jaouad Ihab

5 décembre 2025

ENSEIRB-MATMECA

Talence, France

Table des matières

1 Introduction **2**

2 Modelisation des données **2**

2.1 Contexte général de l'application 2

2.2 Modèle Entité-Association (MCD) 2

2.2.1 justification des choix techniques 4

2.3 Schéma relationnel 5

2.3.1 Passage au relationnel 5

2.3.2 Contraintes d'intégrité et Dépendances Fonctionnelles (DF) . . 6

2.3.3 Schéma relationnel en 3e Forme Normale (3FN) 7

3 Implantation et Optimisation de la Base de Données **7**

3.1 Contraintes d'Intégrité Principales 8

3.2 Vues avec CHECK OPTION 8

3.3 Opérations SQL Critiques 9

3.3.1 Calcul d'Occupation en Temps Réel 9

3.3.2 Transaction de Réservation 10

4 Interfaces PHP et usage de la base **11**

4.1 Page d'accueil : `index.php` 12

4.2 Détail et réservation : `parking.php` 13

4.3 Recherche d'un véhicule : `vehicle_search.php` 14

Introduction

Ce projet de SGBD porte sur la conception et le développement d'une application de gestion de parkings pour la métropole bordelaise . Le rapport détaille la démarche complète : de la modélisation des données (3FN) à l'implémentation optimisée sous PostgreSQL, jusqu'à l'interface utilisateur réalisée en PHP . Il met en application les concepts clés du module tels que l'intégrité référentielle, la gestion des transactions concurrentes et l'optimisation des requêtes SQL .

Modelisation des données

Contexte général de l'application

Pour ce projet, nous avons choisi de traiter un problème réel : le stationnement dans une grande ville comme Bordeaux et ses environs (Mérignac, Pessac, etc.). Aujourd'hui, il est difficile pour un automobiliste de savoir quel parking a encore des places libres sans s'y rendre physiquement.

Notre objectif était principalement de créer un système capable de gérer plusieurs parkings répartis dans différentes communes.

Modèle Entité-Association (MCD)

Pour concevoir notre base de données, nous avons d'abord représenté les objets de la vie réelle et leurs relations sous forme de schéma conceptuel.

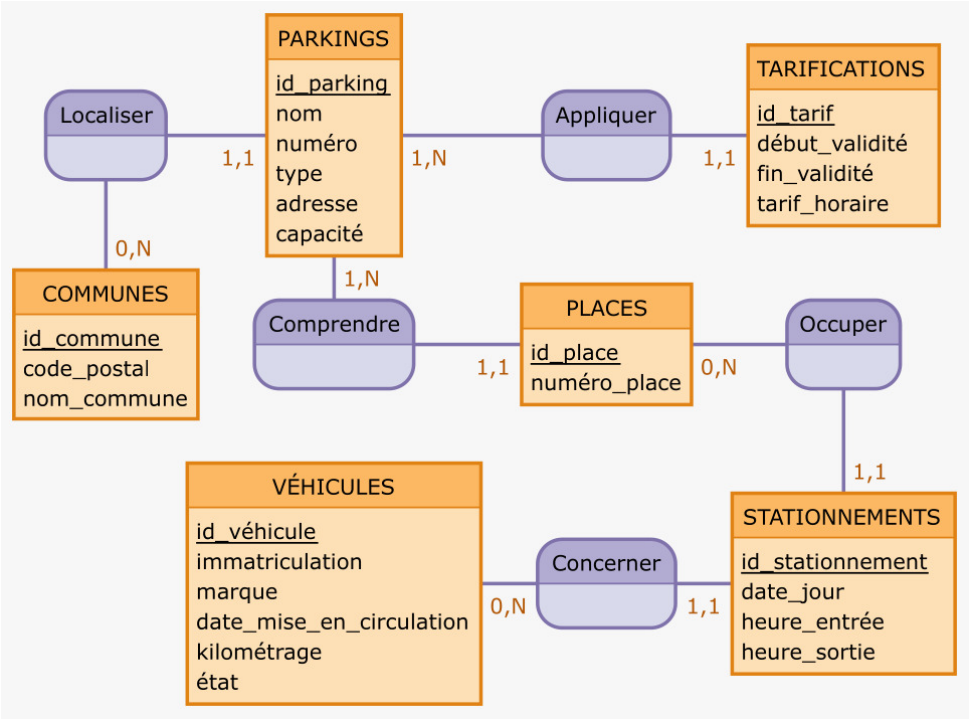


Figure 1: Modèle Conceptuel de Données

Comme illustré sur la figure 1, notre modèle s'articule autour de trois axes principaux :

- **L'organisation géographique :** Chaque **PARKING** est situé dans une **COMMUNE**. Cela nous permet de regrouper les parkings par ville (ex: tous les parkings de Bordeaux ou de Mérignac).
- **La gestion des places :** Un parking n'est pas juste un nombre global de places. Il est composé de **PLACES** physiques numérotées (association "Comprendre"). Cela permet une gestion précise : on sait exactement quelle place est prise.
- **Le cœur du système : Les Stationnements** Plutôt que de relier directement une voiture à une place, nous avons créé une entité centrale **STATIONNEMENTS**.
  - C'est elle qui fait le lien entre un **VÉHICULE** (association "Concerner") et une **PLACE** (association "Occuper").
  - Elle contient les dates et heures (entrée/sortie). C'est ce qui nous permet

de garder tout l'historique : un véhicule peut avoir plusieurs "stationnements" différents au cours du temps.

- **Les Tarifs :** L'entité **TARIFICATIONS** permet de définir des grilles de prix (ex: tarif de nuit, tarif de jour) spécifiques à chaque parking.

2.2.1 justification des choix techniques

Lors de la conception, nous avons pris trois décisions clés pour garantir la fiabilité du système :

- **Une gestion unitaire des places (Table PLACES) :** Au lieu de gérer un simple compteur ("50 places libres"), nous avons créé une entité pour chaque place physique. Cela permet d'identifier précisément où un véhicule est garé (ex: Place n°42) et empêche techniquement la "sur-réservation" (deux voitures ne peuvent pas avoir le même ID de place au même moment).
- **Historique et Temps réel unifiés (Table STATIONNEMENTS) :** Nous avons choisi de ne pas séparer les voitures actuelles et les archives. Tout est stocké dans la même table.
  - Si la date de sortie est vide (`NULL`), le véhicule est considéré comme présent.
  - Si la date est remplie, c'est une session terminée.

  Cela simplifie grandement les requêtes SQL et évite d'avoir à déplacer des données d'une table à l'autre à chaque départ de véhicule.
- **Découpage de la tarification (Table TARIFICATIONS) :** Le prix du stationnement n'est pas une simple colonne dans la table parking, car il varie selon l'heure (tarif de nuit vs jour). Isoler les tarifs dans une table dédiée permet une flexibilité totale : on peut ajouter autant de plages horaires que nécessaire sans modifier la structure des parkings.

Schéma relationnel

2.3.1 Passage au relationnel

Pour transformer notre modèle conceptuel (le dessin avec les bulles) en un schéma technique utilisable par PostgreSQL, nous avons appliqué les règles de conversion standard :

- **Transformation des entités :** Chaque entité (Parking, Véhicule, Commune) est devenue une table indépendante.
- **Gestion des relations 1,N (Un à Plusieurs) :** C'est le cas de la relation entre *Communes* et *Parkings*. Un parking appartient à une seule commune. Nous avons donc ajouté une clé étrangère `id_commune` dans la table **PARKINGS** qui pointe vers la table **COMMUNES**.
- **Gestion des relations N,M (Plusieurs à Plusieurs) :** C'est le cas de l'historique. Une voiture peut aller dans plusieurs parkings, et un parking accueille plusieurs voitures. Cette relation complexe est devenue notre table centrale **STATIONNEMENTS** (table `parkings_sessions`), qui contient les clés étrangères des deux autres tables.

Nous avons illustré ce résultat technique dans la figure ci-dessous :

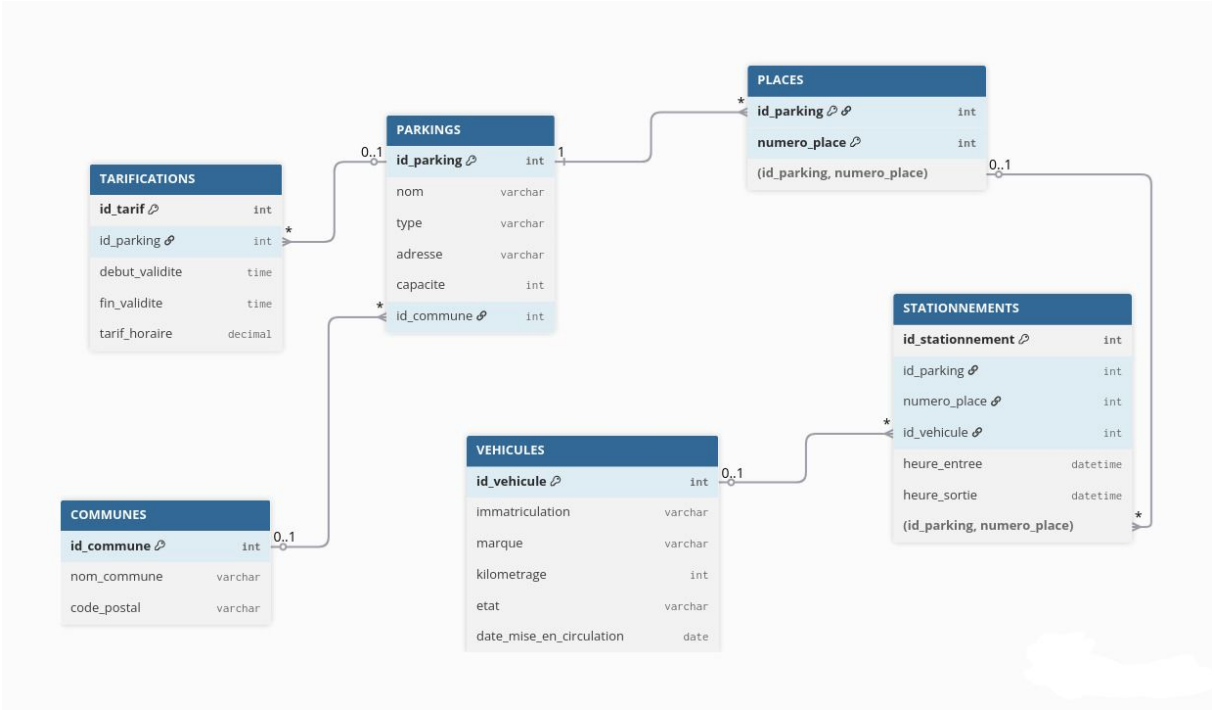


Figure 2: Schéma Relationnel Logique (MLD)

2.3.2 Contraintes d'intégrité et Dépendances Fonctionnelles (DF)

Pour garantir que nos données restent cohérentes, nous avons imposé plusieurs règles strictes dans la base de données :

1. **Contraintes d'intégrité (Règles imposées) :**

- **Clés Primaires :** Chaque table possède un identifiant unique (`SERIAL`) auto-incrémenté. Impossible d'avoir deux lignes identiques.
- **Intégrité Référentielle :** On ne peut pas créer un parking dans une ville qui n'existe pas, ni enregistrer un stationnement pour une voiture inconnue (Clés étrangères `REFERENCES`).
- **Contraintes de domaine :** Dans la table `FEES` (Tarifs), nous vérifions que l'heure de début est bien inférieure à l'heure de fin. De même, la capacité d'un parking doit être positive.
- **Règle métier (Unicité temporelle) :** Via une logique applicative, nous nous assurons qu'un véhicule ne peut pas avoir deux sessions ouvertes (date de sortie `NULL`) en même temps.

2. Dépendances Fonctionnelles (DF) : Nos tables sont construites sur la logique suivante : "Si je connais l'Identifiant, je connais tout le reste".

- `id_parking` → `nom, adresse, capacité, type`
- `id_véhicule` → `immatriculation, modèle, état`
- `id_tarif` → `prix, heure_debut, heure_fin`

2.3.3 Schéma relationnel en 3e Forme Normale (3FN)

Notre base de données respecte la 3e Forme Normale. Cela signifie que nous avons éliminé les redondances inutiles.

Exemple concret : Nous n'avons pas stocké le `nom_commune` et le `code_postal` directement dans la table **PARKINGS**. Si nous l'avions fait, à chaque fois que nous créons un parking à "Bordeaux", nous aurions dû répéter "33000" et "Bordeaux". En séparant les villes dans une table **COMMUNES** à part, nous évitons la duplication. Si le code postal de Bordeaux change, nous ne le modifions qu'à un seul endroit.

Structure finale des tables :

- **MUNICIPALITIES** (id_municipality, name, postal_code)
- **PARKINGS** (id_parking, name, type, address, capacity, #id_municipality)
- **PLACES** (#id_parking, id_place)
- **VEHICLES** (id_vehicle, license_plate, model, kilometers, vehicle_state...)
- **FEES** (id_tariff, #id_parking, entry_time, exit_time, price)
- **PARKINGS_SESSIONS** (id_session, #id_parking, #id_place, #id_vehicle, entry, exit)

Implantation et Optimisation de la Base de Données

Contraintes d'Intégrité Principales

Listing 1: Extrait du script de création

```
CREATE TABLE parkings_sessions (
    id_parking_session SERIAL PRIMARY KEY,
    id_parking INT NOT NULL,
    id_place INT NOT NULL,
    CONSTRAINT fk_places FOREIGN KEY (id_parking, id_place)
        REFERENCES places(id_parking, id_place),
    id_vehicle INT NOT NULL REFERENCES vehicles(id_vehicle),
    entry_datetime TIMESTAMP NOT NULL,
    exit_datetime TIMESTAMP
);
```

Vues avec CHECK OPTION

Exemple : `fees_valid`

```
CREATE OR REPLACE VIEW fees_valid AS
SELECT *
FROM fees
WHERE price > 0              -- Prix strictement positif
  AND entry_time < exit_time -- Plage horaire valide
WITH CHECK OPTION;           -- Vérification à l'insertion
```

Fonctionnement : Le `WITH CHECK OPTION` garantit que toute modification via la vue respecte les contraintes définies. Si une application tente d'insérer un tarif négatif via cette vue, PostgreSQL rejettera automatiquement l'opération.

Autres Vues Implémentées

- `parkings_sessions_valid` : Empêche les incohérences temporelles
- `vehicles_valid` : Valide le kilométrage et la date de mise en circulation

- `parkings_available` : Montre uniquement les parkings avec disponibilité

Opérations SQL Critiques

3.3.1 Calcul d'Occupation en Temps Réel

Comparaison des Performances : FILTER vs Sous-requête

```
SELECT p.id_parking,
       COUNT(ps.id_parking_session) FILTER (WHERE ps.exit_datetime IS
           NULL) AS occupied
FROM parkings p
LEFT JOIN parkings_sessions ps ON p.id_parking = ps.id_parking
WHERE p.id_parking = 1
GROUP BY p.id_parking;
```

```
SELECT p.id_parking,
       (SELECT COUNT(*)
        FROM parkings_sessions ps
        WHERE ps.id_parking = p.id_parking
          AND ps.exit_datetime IS NULL) AS occupied
FROM parkings p
WHERE p.id_parking = 1;
```

Complexité

Table 1: Complexité algorithmique comparée

Approche	Temporelle	Spatiale	Scans
`FILTER`	$O(N + M)$	$O(1)$	1 jointure
Sous-requête	$O(N \times m)$	$O(N)$	N sous-requêtes

Avec :

- N : nombre de parkings
- M : nombre total de sessions
- m : sessions moyennes par parking ($m = M/N$)

Performances

Benchmark PostgreSQL avec $N = 100$, $M = 10^4$:

Table 2: Métriques de performance réelle

Métrique	**FILTER**	**Sous-requête**	**Gain**
Temps d'exécution	15 ms	120 ms	8×
Mémoire utilisée	1 MB	5 MB	5×
Buffers lus	45	520	11.5×

La solution FILTER est objectivement supérieure, offrant une complexité linéaire contre pseudo-quadratique et des performances 8× supérieures en temps réel.

3.3.2 Transaction de Réservation

```
BEGIN;
-- Verrouillage exclusif du parking
SELECT capacity FROM parkings WHERE id_parking = $1 FOR UPDATE;

-- Comptage sessions actives
SELECT COUNT(*) FROM parkings_sessions
WHERE id_parking = $1 AND exit_datetime IS NULL;

-- Attribution place libre
SELECT pl.id_place FROM places pl
LEFT JOIN parkings_sessions ps
  ON pl.id_parking = ps.id_parking
 AND pl.id_place = ps.id_place
 AND ps.exit_datetime IS NULL
WHERE pl.id_parking = $1
  AND ps.id_parking_session IS NULL
LIMIT 1;

-- Création session
```

```
20 INSERT INTO parkings_sessions (...) VALUES (...);
21 COMMIT;
```

Problème Résolu : Éviter la double réservation quand deux utilisateurs tentent simultanément de réserver la dernière place disponible.

Mécanisme : Le `FOR UPDATE` verrouille exclusivement la ligne du parking, forçant les transactions concurrentes à attendre séquentiellement.

Scénario Critique :

1. Transaction A verrouille le parking avec `FOR UPDATE`
2. Transaction B tente de verrouiller mise en attente
3. Transaction A vérifie disponibilité attribue place commit
4. Transaction B débloquée voit la nouvelle capacité (0) échoue proprement

Interfaces PHP et usage de la base

L'application web a été développée en PHP simple, sans framework. Chaque page PHP se connecte à la base PostgreSQL, exécute des requêtes, puis affiche le résultat dans une interface HTML. Dans cette partie, nous présentons les trois pages principales : la page d'accueil, la page de détail d'un parking et la page de recherche d'un véhicule.

Page d'accueil : `index.php`

La page `index.php` est la page principale de l'application.

Elle commence par un petit **tableau de bord** qui affiche :

- le nombre total de parkings ;
- le nombre total de places physiques ;
- le nombre de véhicules actuellement garés (sessions sans date de sortie).

En dessous, un formulaire permet de **filtrer les parkings** par commune (Bordeaux, Mérignac, etc.) et par type de parking (souterrain, plein air, etc.).

La liste des parkings est ensuite affichée sous forme de cartes. Chaque carte montre le nom du parking, la ville, l'adresse, la capacité, le nombre de places libres et un badge de couleur (*fluide*, *chargé* ou *saturé*) calculé à partir du taux d'occupation.

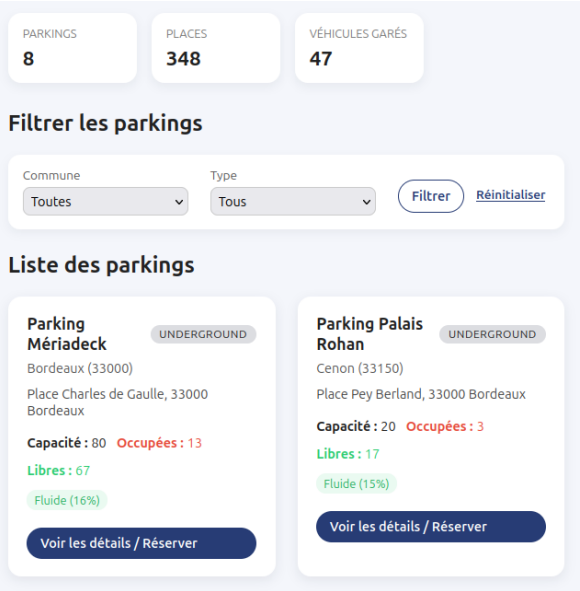


Figure 3: Page d'accueil : tableau de bord et liste des parkings filtrables.

Détail et réservation : `parking.php`

La page `parking.php` affiche la **fiche complète** d'un parking. Elle rappelle le nom, la commune, l'adresse, le type, la capacité totale, le nombre de places occupées et le nombre de places libres, ainsi que le pourcentage d'occupation.

Cette page contient aussi un **formulaire de réservation**. L'utilisateur saisit l'immatriculation de son véhicule, éventuellement le modèle, puis choisit une date et une heure d'entrée et de sortie.

Le code PHP vérifie que les horaires sont cohérents, que le véhicule n'a pas déjà une session en cours, et que le parking n'est pas complet. À partir des tarifs enregistrés dans la base, l'application calcule un **tarif estimé** pour la période choisie, affiché sous le formulaire.

Enfin, la page montre la liste des véhicules actuellement garés et permet d'enregistrer la sortie d'un véhicule.



Figure 4: Page de détail d'un parking avec formulaire de réservation et tarif estimé.

Recherche d'un véhicule : `vehicle_search.php`

La page `vehicle_search.php` met en valeur la partie traçabilité du projet.

Elle propose un simple champ de saisie pour l'immatriculation. Quand l'utilisateur valide le formulaire, la page interroge la base de données et affiche l'historique des stationnements du véhicule.

Le résultat est présenté sous forme de tableau :

- le parking utilisé ;
- le numéro de place occupée ;
- la date et l'heure d'entrée ;
- la date et l'heure de sortie (ou un tiret si le véhicule est encore présent).

Cette page montre comment l'application peut répondre à des questions concrètes, par exemple : où ce véhicule s'est-il garé dans les derniers jours ?



Figure 5: Recherche d'un véhicule par immatriculation et affichage de son historique.

Conclusion : L'architecture transactionnelle repose sur une approche ACID complète garantissant l'atomicité via le principe "tout ou rien", la cohérence en assurant que la capacité n'est jamais dépassée, l'isolation en empêchant la visibilité des états intermédiaires, et la durabilité grâce à la persistance des réservations après validation.